



Aula 08 – App Maui – Listagem de partidas

1. Abra a classe **PartidaService**, em (A) ajuste o endereço da api no Azure conforme demais classes de serviço, e em (B) crie a variável global que vai receber o token, além do construtor para alimentar o token.

```
public class PartidaService : Request
{
    private readonly Request _request;
    private const string _apiUrlBase = "https://xyz.azurewebsites.net/Partidas";
    // (A)

    0 references
    public PartidaService()
    {
        _request = new Request();
    }

    // (B)

    string _token = string.Empty;

    //ctor + TAB: Atalhos para criar construtor
    1 reference
    public PartidaService(string token)
    {
        _token = token;
        _request = new Request();
    }
}
```

2. Ainda em PartidaService ajuste os métodos para ao invés de receber vazio (string.empty), passar a receber a variável global `_token`, que já virá com informações passadas pela viewModel.

```
public async Task<ObservableCollection<Partida>> GetPartidasAsync()
{
    string urlComplementar = string.Format("{0}", "/GetAll");

    ObservableCollection<Partida> lista =
        await _request.GetAsync<ObservableCollection<Partida>>(_apiUrlBase + urlComplementar, _token);

    return lista;
}

0 references
public async Task<Partida> PostPartidaAsync(Partida j)
{
    Partida partidaInserido = await _request.PostAsync<Partida>(_apiUrlBase, j, _token);
    return partidaInserido;
}

1 reference
public async Task<ObservableCollection<PartidaDTO>> GetPartidasDTOAsync()
{
    string urlComplementar = string.Format("{0}", "/ObterTabela");

    ObservableCollection<PartidaDTO> lista =
        await _request.GetAsync<ObservableCollection<PartidaDTO>>(_apiUrlBase + urlComplementar, _token);

    return lista;
}
```



3. Abra a classe chamada **TabelaViewModel** e declare dentro da classe um objeto **PartidaService**, uma propriedade para armazenar a lista de partidas, além de inicializar esses itens no construtor (ctor + TAB). Veja que também iremos recuperar o token das preferences para enviar para a classe de serviço.

```
public class TabelaViewModel : BaseViewModel
{
    PartidaService _partidaService;
    3 references
    public ObservableCollection<PartidaDTO> Partidas { get; set; }
    1 reference
    public TabelaViewModel()
    {
        string token = Preferences.Get("UsuarioToken", string.Empty);
        _partidaService = new PartidaService(token);

        Partidas = new ObservableCollection<PartidaDTO>();
    }

    //Próxima etapa aqui
}
```

4. Crie o método **ObterPartidas** e acione ele antes do final do construtor.

```
public TabelaViewModel()
{
    string token = Preferences.Get("UsuarioToken", string.Empty);
    _partidaService = new PartidaService(token);

    Partidas = new ObservableCollection<PartidaDTO>();
    _ = ObterPartidas();
}

1 reference
public async Task ObterPartidas()
{
    try
    {
        Partidas = await _partidaService.GetPartidasDTOAsync();
        OnPropertyChanged(nameof(Partidas));
    }
    catch (Exception ex)
    {
        await Application.Current.MainPage
            .DisplayAlertAsync("Ops", ex.Message, "Detalhes" + ex.InnerException, "Ok");
    }
}
```



5. Clique com direito na pasta **Views** e crie uma pasta chamada **Partidas**. Clique com direito na pasta Partidas crie uma .Net MAUI/Content Page com o nome **TabelaView.xaml**. Remova o VerticalStackLayout que está dentro da tag ContentPage.Content e adicione o conteúdo abaixo.

```
<ScrollView>
    <VerticalStackLayout Padding="10,0,0,0">
        <CollectionView x:Name="listView" ItemsSource="{Binding Partidas}">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Border Margin="5" Padding="10" BackgroundColor="LightGray">
                        <VerticalStackLayout Spacing="8">
                            <Label FontSize="16" FontAttributes="Bold" HorizontalOptions="Center"
                                Text="{Binding DataHora, StringFormat='{0:dd/MM/yyyy HH:mm}}'" />
                            <Grid ColumnDefinitions="*,Auto">
                                <Label Grid.Column="0" FontSize="18" Text="{Binding TimeMandante}" />
                                <Label Grid.Column="1" FontSize="18" FontAttributes="Bold"
                                    Text="{Binding GolsMandante}" />
                            </Grid>
                            <Grid ColumnDefinitions="*,Auto">
                                <Label Grid.Column="0" FontSize="18" Text="{Binding TimeVisitante}" />
                                <Label Grid.Column="1" FontSize="18" FontAttributes="Bold"
                                    Text="{Binding GolsVisitante}" />
                            </Grid>
                            <BoxView HeightRequest="1" />
                            <Label FontSize="14" Text="{Binding Estadio}" />
                            <Label FontSize="12" Text="{Binding Cidade}" />
                        </VerticalStackLayout>
                    </Border>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</ScrollView>
```

6. Abra a parte de código da View (F7 ou clique direito View Code) e realize a vinculação com a ViewModel. Crie também uma sobrecarga do método OnAppearing. Isso fará com que toda vez que a view aparecer, busque informações na API, e não apenas no primeiro carregamento.

```
public partial class TabelaView : ContentPage
{
    TabelaViewModel viewModel;
    0 references
    public TabelaView()
    {
        InitializeComponent();

        viewModel = new TabelaViewModel();
        BindingContext = viewModel;
        Title = "Tabela";
    }

    0 references
    protected override void OnAppearing()
    {
        base.OnAppearing();
        _ = viewModel.ObterPartidas();
    }
}
```



7. Abra **AppShell.xaml** para criar um menu para a view

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
    x:Class="AppLibertadoresHAS.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:AppLibertadoresHAS"
    xmlns:partidas="clr-namespace:AppLibertadoresHAS.Views.Partidas"
    Title="AppLibertadoresHAS">

    <ShellContent
        Title="Home"
        ContentTemplate="{DataTemplate local:MainPage}" Route="MainPage" />

    <ShellContent
        Title="Partidas"
        ContentTemplate="{DataTemplate partidas:TabelaView}" Route="tabela" />

</Shell>
```

• Resultado esperado

The screenshot shows a mobile application window with a hamburger menu icon on the left. The title bar of the window says "Tabela". The table displays football matches with columns for date and time, team names, and scores.

08/04/2026 21:00	
Sporting Cristal	2
Cerro Porteño	2
Monumental U Lima	
08/04/2026 21:30	
Palmeiras	2
Atlético Nacional	0
Monumental U Lima	
09/04/2026 18:00	
Libertad	1
Olimpia	0
Olimpico de la UCV Caracas	
09/04/2026 19:00	
Rosario Central	1
Independiente del Valle	1
Gigante de Arroyito Rosário	
09/04/2026 21:00	
Platense	0
Corinthians	2
Ciudad de Vicente López Buenos Aires	